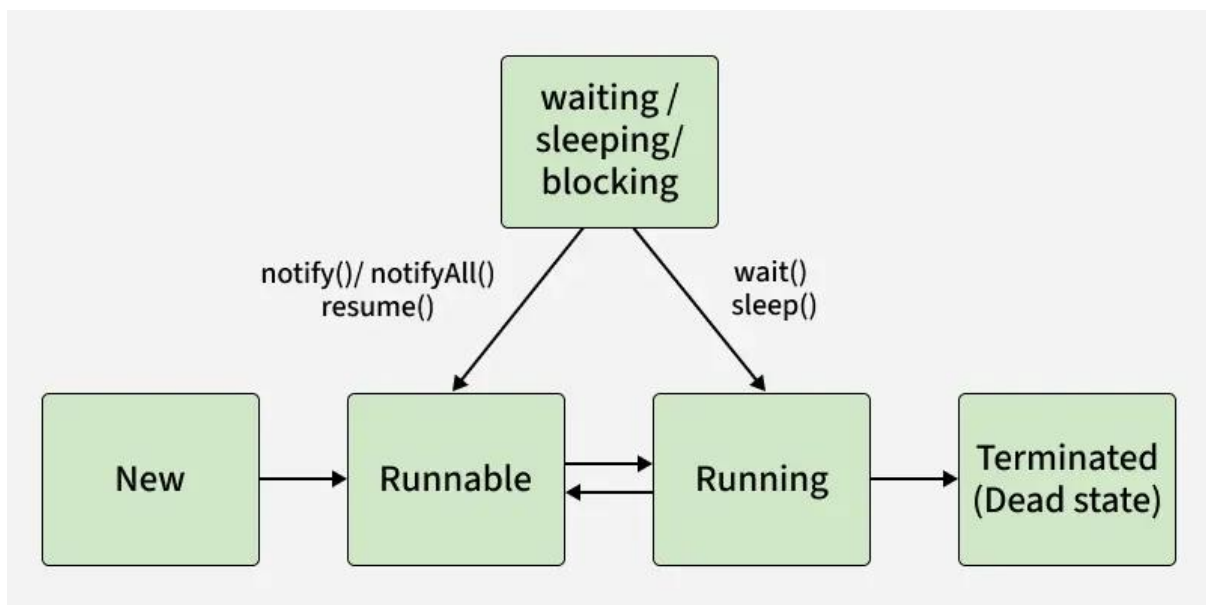


Multithreading

Thread Life Cycle :-

- A thread in Java can exist in any one of the following states at any given time.
- A thread lies only in one of the shown states at any instant:
 1. New State
 2. Runnable State
 3. Blocked State
 4. Waiting State
 5. Timed Waiting State
 6. Terminated State



1. New State:

- When a thread is created, it is in the new state.
- It hasn't started execution yet.

2. Runnable State:

- The thread is ready to run or is running.
- The thread scheduler decides when it will execute.

3. Blocked State:

- The thread is waiting to acquire a lock.
- It moves to runnable state once the lock is available.

4. Waiting State:

- The thread waits indefinitely using `wait()` or `join()`.
- It resumes when notified by another thread.

5. Timed Waiting State:

- The thread waits for a specified time period.
- It automatically resumes after timeout or notification.

6. Terminated State:

- The thread's execution has finished.
- It may end normally or due to an error.

Running Threads :-

- There are two methods used for running Threads in Java:

1) run() Method:

- Contains the code for the thread.
- Calling it directly behaves like a normal method call.

2) start() Method:

- Launches a new thread and internally calls run() concurrently.

Example –

```
package multithreading;

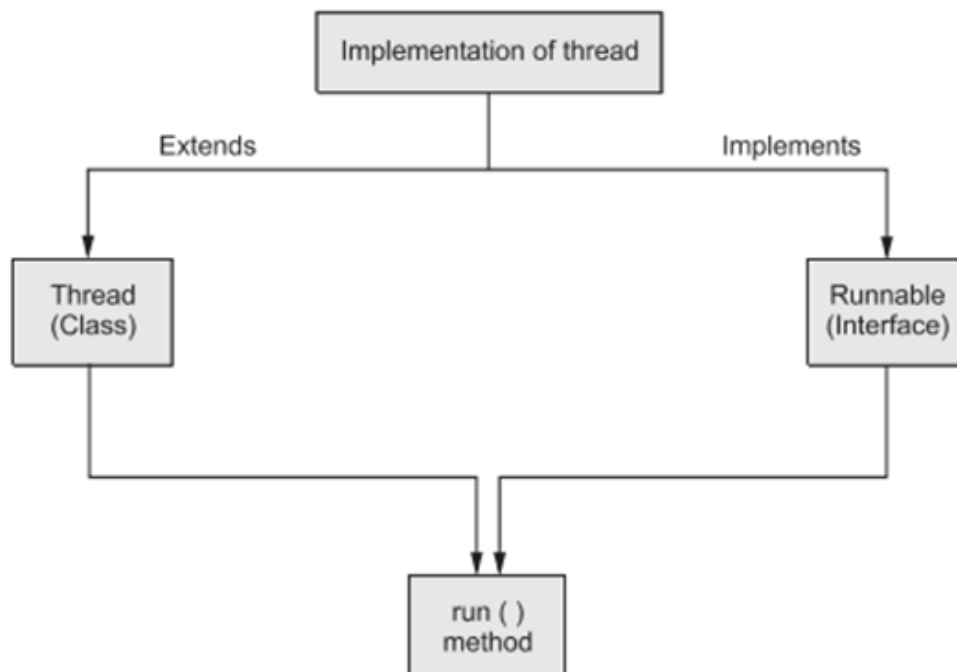
public class first extends Thread {
    public void run()
    { //RUNNING
        System.out.println("Thread is Running");
        try {
            Thread.sleep(5000);
        } catch (InterruptedException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
        System.out.println("Executed!");
    }
    public static void main(String[] args) {
        first th1=new first(); //NEW
        System.out.println(th1.getState()); //RUNNABLE
        th1.start();
        System.out.println(th1.getState()); //RUNNABLE
        System.out.println("is alive for first time: "+th1.isAlive());
        System.out.println("is alive for second time: "+th1.isAlive());
        System.out.println(th1.getState()); //TERMINATED
    }
}
```

Output –

```
NEW
RUNNABLE
Thread is Running
is alive for first time: true
is alive for second time: true
TIMED_WAITING
Executed!
```

How to achieve Multithreading ?

- Multithreading is achieved by creating and running multiple threads.
- There are two main ways to create threads in Java:
 1. By extending the Thread class
 2. By implementing the Runnable interface
- Each thread runs its code inside the run() method.
- To start a thread, call the start() method, not run().
- The JVM and CPU scheduler manage thread execution automatically .



2. Runnable Interface :-

- The Runnable interface is used to create a thread task.
- It has a single method run() where the thread code is written.
- A Thread object executes it by calling start().

Steps to Achieve Multithreading using Runnable Interface –

- 1) Create a class that implements Runnable.
- 2) Override the run() method to define the thread's task.
- 3) Create a Thread object and pass the Runnable instance to it.
- 4) Call the start() method on the Thread object to begin execution.

Example –

```
package multithreading;

public class second implements Runnable{
    @Override
    public void run()
    {
        System.out.println("Task is Running");
    }
    public static void main(String[] args) {
        second s1=new second();    //NEW
        Thread t1=new Thread(s1);  //creating the object of thread
        t1.start();
    }
}
```

Output –

Task is Running